

function f or a predicate p . If f takes two arguments (for example, $f(x, y)$), $\mathbf{D}_{\text{in}}(f)$ returns two domains, one per argument.

- $\mathbf{D}_{\text{out}} : \mathcal{F} \rightarrow \mathcal{D}$. Intuitively, $\mathbf{D}_{\text{out}}(f)$ returns the range of a function symbol.

Real Logic may also contain propositional variables, as follows: if P is a 0-ary predicate with $\mathbf{D}_{\text{in}}(P) = \langle \rangle$ (the empty sequence of domains) then P is a propositional variable (an atom with truth-value in the interval $[0, 1]$).

A term is constructed recursively in the usual way from constant symbols, variables, and function symbols. An expression formed by applying a predicate symbol to an appropriate number of terms with appropriate domains is called an atomic formula, which evaluates to *true* or *false* in classical logic and a number in $[0, 1]$ in the case of Real Logic. We define the set of terms of the language as follows:

- each element t of $\mathcal{X} \cup \mathcal{C}$ is a term of the domain $\mathbf{D}(t)$;
- if t_i is a term of domain $\mathbf{D}(t_i)$ for $1 \leq i \leq n$ then $t_1 t_2 \dots t_n$ (the sequence composed of t_1 followed by t_2 and so on, up to t_n) is a term of the domain $\mathbf{D}(t_1)\mathbf{D}(t_2) \dots \mathbf{D}(t_n)$;
- if t is a term of the domain $\mathbf{D}_{\text{in}}(f)$ then $f(t)$ is a term of the domain $\mathbf{D}_{\text{out}}(f)$.

We allow the following set of formula in $\mathcal{L}$:

- $t_1 = t_2$ is an atomic formula for any terms t_1 and t_2 with $\mathbf{D}(t_1) = \mathbf{D}(t_2)$;
- $p(t)$ is an atomic formula if $\mathbf{D}(t) = \mathbf{D}_{\text{in}}(p)$;
- If ϕ and ψ are formula and $x_1, \dots, x_n$ are n distinct variable symbols then $\diamond\phi$, $\phi \circ \psi$ and $Qx_1 \dots x_n \phi$ are formula, where $\diamond$ is a unary connective, $\circ$ is a binary connective and Q is a quantifier.

We use $\diamond \in \{\neg\}$ (negation), $\circ \in \{\wedge, \vee, \rightarrow, \leftrightarrow\}$ (conjunction, disjunction, implication and bi-conditional, respectively) and $Q \in \{\forall, \exists\}$ (universal and existential, respectively).

Example 1. Let **Town** denote the domain of towns in the world and **People** denote the domain of living people. Suppose that $\mathcal{L}$ contains the constant symbols Alice, Bob and Charlie of domain **People**, and Rome and Seoul of domain **Town**. Let x be a variable of domain **People** and u be a variable of domain **Town**. The term x, u (i.e. the sequence x followed by u) has domain **People, Town** which denotes the Cartesian product between **People** and **Town** ($\mathbf{D}(x, u) = \mathbf{D}(x) \times \mathbf{D}(u)$). Alice, Rome is interpreted as an element of the domain **People, Town**. Let *lives_in* be a predicate with input domain $\mathbf{D}_{\text{in}}(\text{lives_in}) = \mathbf{D}(x, u)$. *lives_in*(Alice, Rome) is a well-formed expression, whereas *lives_in*(Bob, Charlie) is not.

2.2. Semantics of Real Logic

The semantics of Real Logic departs from the standard abstract semantics of First-order Logic (FOL). In Real Logic, domains are interpreted concretely by tensors in the real field.¹ Every object denoted by constants, variables, and terms, is interpreted as a tensor of real values. Functions are

¹In the rest of the paper, we commonly use "tensor" to designate "tensor in the real field".

interpreted as real functions or tensor operations. Predicates are interpreted as functions or tensor operations projecting onto a value in the interval $[0, 1]$.

To emphasize the fact that in Real Logic symbols are grounded onto real-valued features, we use the term *grounding*, denoted by $\mathcal{G}$, in place of *interpretation*². Notice that this is different from the common use of the term *grounding* in logic, which indicates the operation of replacing the variables of a term or formula with constants or terms containing no variables. To avoid confusion, we use the synonym *instantiation* for this purpose. $\mathcal{G}$ associates a tensor of real numbers to any term of $\mathcal{L}$, and a real number in the interval $[0, 1]$ to any formula ϕ of $\mathcal{L}$. Intuitively, $\mathcal{G}(t)$ are the numeric features of the objects denoted by t , and $\mathcal{G}(\phi)$ represents the system's degree of confidence in the truth of ϕ ; the higher the value, the higher the confidence.

2.2.1. Grounding domains and the signature

A grounding for a logical language $\mathcal{L}$ on the set of domains $\mathcal{D}$ provides the interpretation of both the domain symbols in $\mathcal{D}$ and the non-logical symbols in $\mathcal{L}$.

Definition 1. A grounding $\mathcal{G}$ associates to each domain $D \in \mathcal{D}$ a set $\mathcal{G}(D) \subseteq \bigcup_{n_1 \dots n_d \in \mathbb{N}^*} \mathbb{R}^{n_1 \times \dots \times n_d}$. For every $D_1 \dots D_n \in \mathcal{D}^*$, $\mathcal{G}(D_1 \dots D_n) = \times_{i=1}^n \mathcal{G}(D_i)$, that is $\mathcal{G}(D_1) \times \mathcal{G}(D_2) \times \dots \times \mathcal{G}(D_n)$.

Notice that the elements in $\mathcal{G}(D)$ may be tensors of any rank d and any dimensions $n_1 \times \dots \times n_d$, as $\mathbb{N}^*$ denotes the Kleene star of $\mathbb{N}$.³

Example 2. Let `digit_images` denote a domain of images of handwritten digits. If we use images of 256×256 RGB pixels, then $\mathcal{G}(\text{digit_images}) \subseteq \mathbb{R}^{256 \times 256 \times 3}$. Let us consider the predicate `is_digit`(, 8). The terms , 8 have domains `digit_images`, `digits`. Any input to the predicate is a tuple in $\mathcal{G}(\text{digit_images}, \text{digits}) = \mathcal{G}(\text{digit_images}) \times \mathcal{G}(\text{digits})$.

A grounding assigns to each constant symbol c , a tensor $\mathcal{G}(c)$ in the domain $\mathcal{G}(\mathbf{D}(c))$; It assigns to a variable x a finite sequence of tensors $d_1 \dots d_k$, each in $\mathcal{G}(\mathbf{D}(x))$. These tensors represent the instances of x . Differently from in FOL where a variable is assigned to a single value of the domain of interpretations at a time, in Real Logic a variable is assigned to a sequence of values in its domain, the k examples of x . A grounding assigns to a function symbol f a function taking tensors from $\mathcal{G}(\mathbf{D}_{\text{in}}(f))$ as input, and producing a tensor in $\mathcal{G}(\mathbf{D}_{\text{out}}(f))$ as output. Finally, a grounding assigns to a predicate symbol p a function taking tensors from $\mathcal{G}(\mathbf{D}_{\text{in}}(p))$ as input, and producing a truth-value in the interval $[0, 1]$ as output.

Definition 2. A *grounding* $\mathcal{G}$ of $\mathcal{L}$ is a function defined on the signature of $\mathcal{L}$ that satisfies the following conditions:

1. $\mathcal{G}(x) = \langle d_1 \dots d_k \rangle \in \times_{i=1}^k \mathcal{G}(\mathbf{D}(x))$ for every variable symbol $x \in \mathcal{X}$, with $k \in \mathbb{N}_0^+$. Notice that $\mathcal{G}(x)$ is a sequence and not a set, meaning that the same value of $\mathcal{G}(\mathbf{D}(x))$ can occur multiple times in $\mathcal{G}(x)$, as is usual in a Machine Learning data set with “attributes” and “values”;

²An interpretation is an assignment of truth-values *true* or *false*, or in the case of Real Logic a value in $[0, 1]$, to a formula. A model is an interpretation that maps a formula to *true*

³A tensor of rank 0 corresponds to a scalar, a tensor of rank 1 to a vector, a tensor of rank 2 to a matrix and so forth, in the usual way.

2. $\mathcal{G}(f) \in \mathcal{G}(\mathbf{D}_{\text{in}}(f)) \rightarrow \mathcal{G}(\mathbf{D}_{\text{out}}(f))$ for every function symbol $f \in \mathcal{F}$;
3. $\mathcal{G}(p) \in \mathcal{G}(\mathbf{D}_{\text{in}}(p)) \rightarrow [0, 1]$ for every predicate symbol $p \in \mathcal{P}$.

If a grounding depends on a set of parameters θ , we denote it as $\mathcal{G}_\theta(\cdot)$ or $\mathcal{G}(\cdot \mid \theta)$ interchangeably. Section 4 describes how such parameters can be learned using the concept of satisfiability.

2.2.2. Grounding terms and atomic formulas

We now extend the definition of grounding to all first-order terms and atomic formulas. Before formally defining these groundings, we describe on a high level what happens when grounding terms that contain free variables.⁴

Let x be a variable that denotes people. As explained in Definition 2, x is grounded as an explicit sequence of k instances ($k = |\mathcal{G}(x)|$). Consequently, a term $\text{height}(x)$ is also grounded in k height values, each corresponding to one instance. We can generalize to expressions with multiple free variables, as shown in Example 3.

In the formal definition below, instead of considering a single term at a time, it is convenient to consider sequences of terms $\mathbf{t} = t_1 t_2 \dots t_k$ and define the grounding on $\mathbf{t}$ (with the definition of the grounding of a single term being derived as a special case). The fact that the sequence of terms $\mathbf{t}$ contains n distinct variables $x_1, \dots, x_n$ is denoted by $\mathbf{t}(x_1, \dots, x_n)$. The grounding of $\mathbf{t}(x_1, \dots, x_n)$, denoted by $\mathcal{G}(\mathbf{t}(x_1, \dots, x_n))$, is a tensor with n corresponding axes, one for each free variable, defined as follows:

Definition 3. Let $\mathbf{t}(x_1, \dots, x_n)$ be a sequence $t_1 \dots t_m$ of m terms containing n distinct variables $x_1, \dots, x_n$. Let each term t_i in $\mathbf{t}$ contain n_i variables $x_{j_{i1}}, \dots, x_{j_{in_i}}$.

- $\mathcal{G}(\mathbf{t})$ is a tensor with dimensions $(|\mathcal{G}(x_1)|, \dots, |\mathcal{G}(x_n)|)$ such that the element of this tensor indexed by $k_1, \dots, k_n$, written as $\mathcal{G}(\mathbf{t})_{k_1 \dots k_n}$, is equal to the concatenation of $\mathcal{G}(t_i)_{k_{j_{i1}} \dots k_{j_{in_i}}}$ for $1 \leq i \leq m$;
- $\mathcal{G}(f(\mathbf{t}))_{i_1 \dots i_n} = \mathcal{G}(f)(\mathcal{G}(\mathbf{t})_{i_1 \dots i_n})$, i.e. the element-wise application of $\mathcal{G}(f)$ to $\mathcal{G}(\mathbf{t})$;
- $\mathcal{G}(p(\mathbf{t}))_{i_1 \dots i_n} = \mathcal{G}(p)(\mathcal{G}(\mathbf{t})_{i_1 \dots i_n})$, i.e. the element-wise application of $\mathcal{G}(p)$ to $\mathcal{G}(\mathbf{t})$.

If term t_i contains n_i variables $x_{j_1}, \dots, x_{j_{n_i}}$ selected from $x_1, \dots, x_n$ then $\mathcal{G}(t_i)_{k_{j_1} \dots k_{j_{n_i}}}$ can be obtained from $\mathcal{G}(\mathbf{t})_{i_1 \dots i_n}$ with an appropriate mapping of indices i to k .

⁴We assume the usual syntactic definition of free and bound variables in FOL. A variable is free if it is not bound by a quantifier ($\forall, \exists$).